IMPLEMENTATION OF LONG SHORT-TERM MEMORY (LSTM) FOR SENTIMENT ANALYSIS

Objective: Develop an LSTM model that reads customer reviews and classifies them as Positive or Negative. Understand the input gate, forget gate, output gate, cell state and hidden state.

1. Execution Workflow

Stage	Description
Input dataset	Load customer reviews and sentiment labels from sentiment_dataset.csv.
Text preprocessing	Convert reviews into token sequences and pad or truncate them to a fixed length.
Embedding layer	Convert token IDs into dense numerical vectors.
LSTM layer	Use gates and a cell state to process the sequence and retain useful information.
Output layer	Use sigmoid activation to estimate the probability of Positive sentiment.
Evaluation	Calculate accuracy, precision, recall and F1-score; inspect confusion matrix and loss curves.

2. Software Requirements

Use Python with TensorFlow/Keras, pandas, NumPy, scikit-learn, Matplotlib and Seaborn. CPU execution is sufficient.

```
python -m pip install --upgrade pip
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn
```

Verify TensorFlow installation:

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

3. Dataset Preparation

Use the previously created **sentiment_dataset.csv**. It should contain columns named **Review** and **Sentiment**. Positive reviews are encoded as 1 and Negative reviews as 0.

Review	Sentiment
The product is excellent	Positive
Amazing quality and fast delivery	Positive
Very poor quality	Negative
The service was terrible	Negative

Dataset limitation: The 100-review synthetic dataset is useful for demonstrating execution, but it is too small and repetitive for reliable real-world model evaluation.

4. Understanding the LSTM Architecture

An LSTM processes a sequence one token at a time. It uses gates to control what information should be forgotten, added to memory and passed forward.

Component	Function
Forget gate	Decides which information from the previous cell state should be retained or removed.
Input gate	Controls which new information should be added to the cell state.
Cell state	Carries important information across the sequence.
Output gate	Controls which information from the cell state is used to produce the current output.
Hidden state	Carries the current output information to the next time step.

LSTM equations

Forget gate: f(t) = sigmoid(Wf [h(t-1), x(t)] + bf)

Input gate: i(t) = sigmoid(Wi [h(t-1), x(t)] + bi)

Candidate cell: C~(t) = tanh(Wc [h(t-1), x(t)] + bc)

Cell update: C(t) = f(t) * C(t-1) + i(t) * C~(t)

Output gate: o(t) = sigmoid(Wo [h(t-1), x(t)] + bo)

Hidden state: h(t) = o(t) * tanh(C(t))

Here, x(t) is the current input, h(t-1) is the previous hidden state, C(t-1) is the previous cell state, W terms are weight matrices, b terms are bias values, sigmoid controls gate values and tanh creates candidate information.

How the gates work: The forget gate filters old memory, the input gate selects new memory, the cell state is updated, and the output gate determines the hidden state. Keras performs these operations internally when the LSTM layer is used.

5. Complete Python Implementation (Part 1)

Create a file named **lstm_sentiment_analysis.py** in the same folder as **sentiment_dataset.csv**. Add the following code.

```
# IMPLEMENTATION OF LSTM FOR SENTIMENT ANALYSIS
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
X = data["Review"].fillna("").astype(str).to_numpy()

# Convert Positive = 1 and Negative = 0
y = (
    data["Sentiment"].str.strip().str.lower()
    .map({"negative": 0, "positive": 1})
)
if y.isna().any():
    raise ValueError("Invalid sentiment labels found.")
y = y.to_numpy(dtype="float32")

# Set random seeds
np.random.seed(42)
tf.random.set_seed(42)

# STEP 2: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# STEP 3: Text preprocessing
MAX_WORDS = 3000
MAX_LENGTH = 40
train_text = tf.constant(X_train, dtype=tf.string)
test_text = tf.constant(X_test, dtype=tf.string)

vectorizer = tf.keras.layers.TextVectorization(
    max_tokens=MAX_WORDS,
    standardize="lower_and_strip_punctuation",
    split="whitespace",
    output_mode="int",
    output_sequence_length=MAX_LENGTH
)

# Learn vocabulary using training data only
vectorizer.adapt(train_text)
vocab_size = len(vectorizer.get_vocabulary())
print("Vocabulary Size:", vocab_size)
```

5. Complete Python Implementation (Part 2)

Continue the same Python file:

```
# STEP 4: Build LSTM architecture
model = tf.keras.Sequential([
    tf.keras.Input(shape=(), dtype=tf.string),
    vectorizer,

    # Embedding layer
    tf.keras.layers.Embedding(
        input_dim=vocab_size,
        output_dim=64,
        mask_zero=True
    ),

    # LSTM layer
    tf.keras.layers.LSTM(
        units=64,
        activation="tanh",
        recurrent_activation="sigmoid",
        dropout=0.2,
        return_sequences=False
    ),

    # Output layer
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# STEP 5: Compile model
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("\\nLSTM MODEL ARCHITECTURE")
model.summary()

# STEP 6: Train model
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

print("\\nTraining LSTM model...")
history = model.fit(
    train_text,
    y_train,
    epochs=20,
    batch_size=8,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
print("\\nTraining completed!")
```

5. Complete Python Implementation (Part 3)

Continue the same Python file:

```
# STEP 7: Evaluate model
test_loss, test_accuracy = model.evaluate(
    test_text, y_test, verbose=0
)
print("\\nLSTM MODEL PERFORMANCE")
print("Test Loss:", round(test_loss, 4))
print("Accuracy:", round(test_accuracy, 4))

# STEP 8: Make predictions
probabilities = model.predict(test_text).ravel()
predictions = (probabilities >= 0.5).astype(int)

print("Precision:", precision_score(
    y_test, predictions, zero_division=0
))
print("Recall:", recall_score(
    y_test, predictions, zero_division=0
))
print("F1 Score:", f1_score(
    y_test, predictions, zero_division=0
))

print("\\nClassification Report:")
print(classification_report(
    y_test, predictions,
    labels=[0, 1],
    target_names=["Negative", "Positive"],
    zero_division=0
))

# STEP 9: Confusion matrix
cm = confusion_matrix(y_test, predictions, labels=[0, 1])
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)
plt.title("LSTM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# STEP 10: Plot training loss
plt.figure(figsize=(7, 4))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("LSTM Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

5. Complete Python Implementation (Part 4)

Add the final prediction and model-saving code to the same Python file:

```
# STEP 11: Predict new reviews
new_reviews = tf.constant([
    "This product is excellent and amazing",
    "The product is very poor and disappointing"
], dtype=tf.string)

new_probabilities = model.predict(new_reviews).ravel()
print("\\nNEW REVIEW PREDICTIONS")

for review, probability in zip(new_reviews.numpy(), new_probabilities):
    review = review.decode("utf-8")
    sentiment = "Positive" if probability >= 0.5 else "Negative"
    print("\\nReview:", review)
    print("Positive Probability:", round(float(probability), 4))
    print("Predicted Sentiment:", sentiment)

# STEP 12: Save trained model
model.save("lstm_sentiment_model.keras")
print("\\nLSTM model saved successfully!")
```

The complete program is now ready to execute.

6. How to Execute the Program

1. Create a folder named **LSTM_Sentiment_Analysis**.

2. Place **sentiment_dataset.csv** and **lstm_sentiment_analysis.py** inside the folder.

3. Open Command Prompt or the VS Code terminal.

4. Navigate to the project folder using the cd command.

5. Install TensorFlow and the required libraries.

6. Run the Python file and observe the training and evaluation outputs.

Terminal commands

```
cd path\to\LSTM_Sentiment_Analysis
python -m pip install --upgrade pip
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn
python lstm_sentiment_analysis.py
```

7. Important LSTM Parameters

Parameter	Explanation
MAX_WORDS = 3000	Maximum vocabulary size retained by TextVectorization.
MAX_LENGTH = 40	Each review is padded or truncated to 40 tokens.
Embedding output_dim = 64	Each token is represented by a trainable 64-dimensional vector.
LSTM units = 64	The recurrent layer has 64 LSTM units.
activation = tanh	Activation used for candidate cell and cell-state output calculations.
recurrent_activation = sigmoid	Activation used by the input, forget and output gates.
dropout = 0.2	Drops a fraction of input connections during training to help reduce overfitting.
Dense(1, sigmoid)	Produces one probability for binary sentiment classification.
epochs = 20	Maximum number of training passes; early stopping may stop sooner.
batch_size = 8	Number of training examples processed before an optimizer update.

8. Understanding the Results

Output	Meaning
Accuracy	Fraction of test reviews classified correctly.
Precision	Of reviews predicted as a class, the fraction that truly belongs to that class.
Recall	Of actual reviews in a class, the fraction correctly identified.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Counts of correct and incorrect predictions for Negative and Positive classes.
Training loss	Error value on training data as the model learns.
Validation loss	Error value on validation data; useful for observing overfitting.

Expected outputs: Model summary, epoch-by-epoch training progress, test metrics, new-review predictions, confusion matrix and training/validation loss graph. The trained model is saved as **lstm_sentiment_model.keras**. Numeric results depend on actual execution.

9. RNN versus LSTM Comparison

Simple RNN	LSTM
Uses a basic hidden state.	Uses both a hidden state and a cell state.
Has a simpler recurrent structure.	Uses input, forget and output gates.
Can struggle with long-term dependencies.	Designed to better learn long-term dependencies.
More susceptible to vanishing gradients.	Helps mitigate the vanishing-gradient problem.

10. Suggested Laboratory Experiment

Title: Implementation of Long Short-Term Memory (LSTM) for Sentiment Analysis.

Aim: To implement an LSTM model for sentiment classification and understand the roles of input gate, forget gate, output gate, cell state and hidden state in sequential text processing.

Learning Outcomes

1. Understand the architecture of LSTM.
2. Explain the functions of input, forget and output gates.
3. Understand the difference between cell state and hidden state.
4. Implement LSTM using Python and TensorFlow.
5. Evaluate sentiment classification using standard performance metrics.

11. Suggested Teaching Sequence

Order	Teaching activity
1	Introduce the basic RNN architecture and its limitations.
2	Explain the input, forget and output gates.
3	Explain cell state and hidden state with an example sentence.
4	Execute the LSTM Python implementation.
5	Discuss evaluation metrics and compare LSTM with the RNN experiment.

Dataset recommendation: The 100-review synthetic dataset is for demonstration. For meaningful evaluation, use a larger, diverse and independently labelled corpus. A small dataset can produce unstable test metrics and overfitting.

12. Official Documentation

TensorFlow installation: tensorflow.org/install/pip
Keras LSTM layer: keras.io - LSTM
Keras TextVectorization: keras.io - TextVectorization